

第3回: 復習回 (Web基礎)

LLMアプリケーション基礎

今日のゴール

来週からのコーディング再開に向けて、前コースで扱った **Web基礎** (FastAPI / fetch / SQLite) を思い出す

今日の流れ

前半

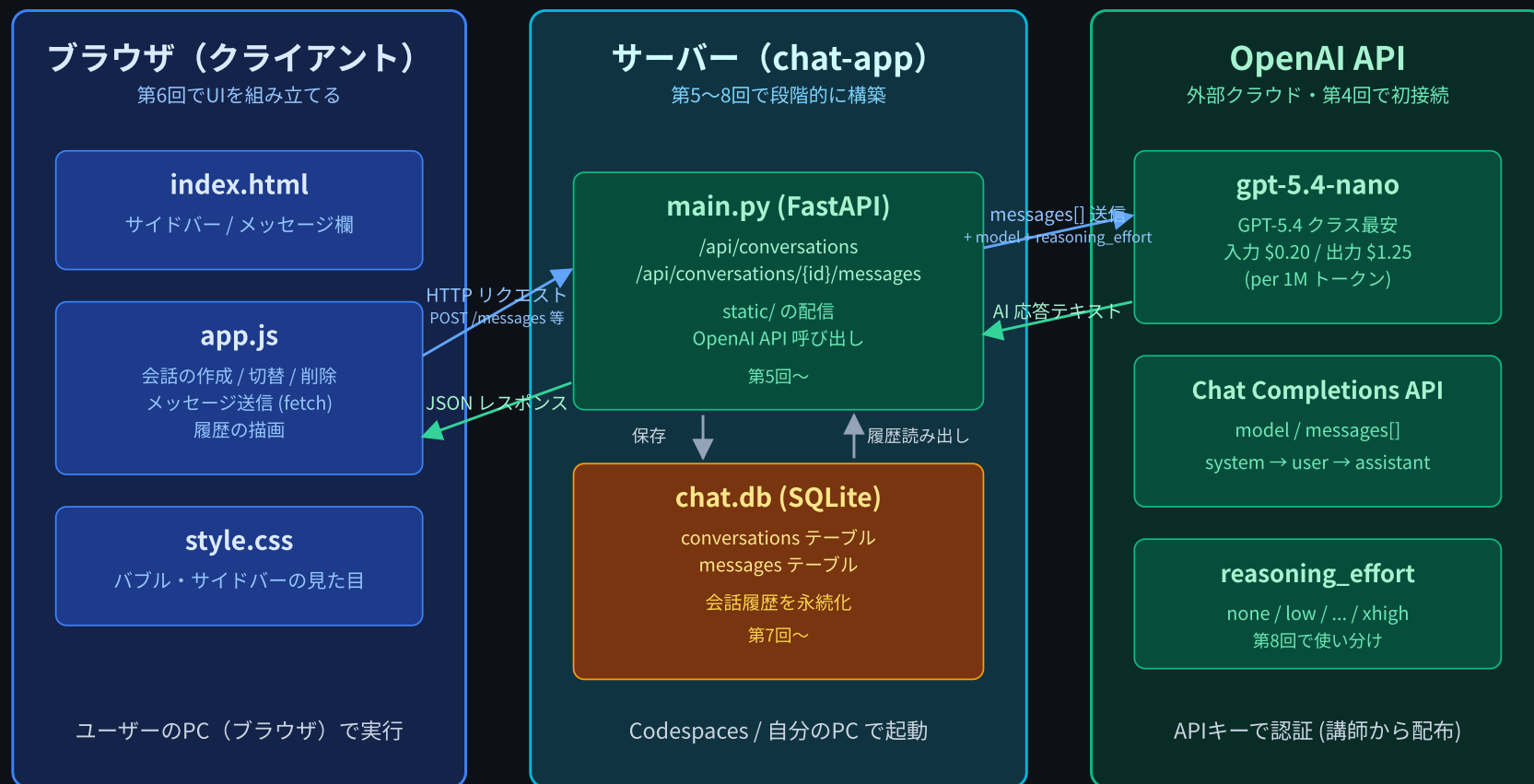
- 開発環境の起動確認
- FastAPI のおさらい（最小Hello / POST + Pydantic）
- フロントからの fetch / async-await

後半

- SQLite のおさらい（`sqlite3` モジュール）
- ミニ演習: `/api/echo` を作って fetch で呼ぶ

LLM / OpenAI には今日はまだ触りません。 純粋にWeb基礎の復習に集中する回です。

chat-app の全体像

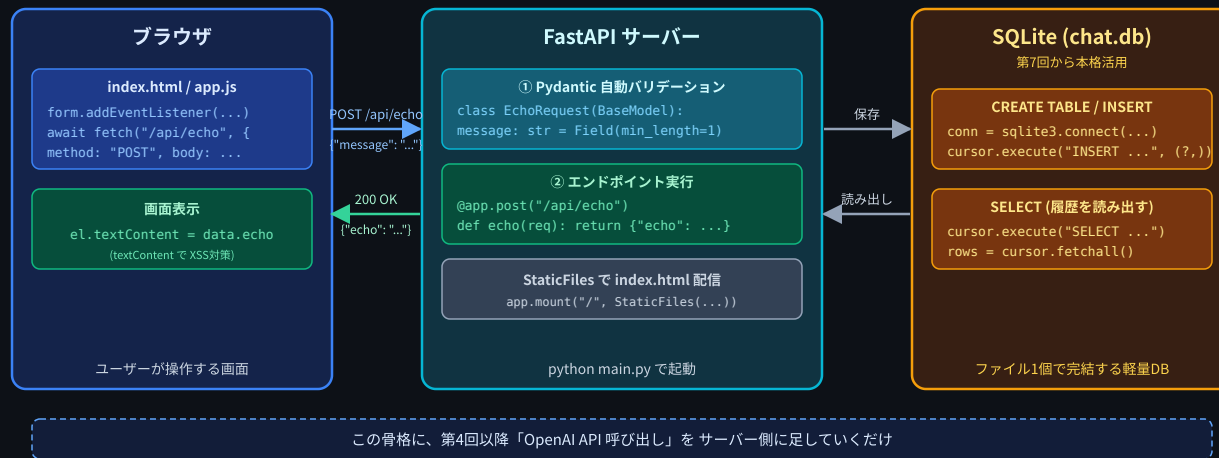


今日は右(OpenAI)には触れず、左(ブラウザ)と中央(FastAPI/SQLite)の骨格を復習する

なぜ今、復習回？

- 第1回・第2回は コードを書かずに LLMの世界観とプロンプトを学んだ
- 第4回からはガッツリ Python / FastAPI / JavaScript を書く
- 「あれ、fetch ってどう書くんだっけ？」を 今日のうちに 解消しておく

ブラウザ ⇄ FastAPI ⇄ SQLite の流れ (echo API)



前コースの TODOアプリと 構造はほぼ同じ。真ん中の処理がCRUDからLLM呼び出しに変わるだけ

前半: FastAPI と fetch を思い出す

0. 開発環境の起動確認

Codespaces / devcontainer を立ち上げて、以下が動くことを確認:

```
# Pythonバージョン  
python --version  
  
# fastapi/uvicorn がインストールされているか  
python -c "import fastapi; print(fastapi.__version__)"
```

動かない場合は devcontainer のリビルド、または

```
pip install -r requirements.txt
```

を試す。

1. FastAPI のおさらい

FastAPI は **Python**製のWebフレームワーク。

- 関数にデコレータ(`@app.get(...)` など)を付けるだけでAPIになる
- リクエスト/レスポンスの型を **Pydantic** で書く
- 自動で Swagger UI (`/docs`) を生成してくれる

最小の Hello World

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def hello():
    return {"message": "Hello, World!"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

起動:

```
python main.py
```

`http://localhost:8000/` にアクセスすると JSON が返る。

HTTPメソッドの復習

メソッド	用途	例
GET	取得	一覧を取る、1件取る
POST	新規作成 / 処理実行	メッセージを送る
PUT	更新	完了状態を切り替える
DELETE	削除	1件削除

FastAPI では:

```
@app.get("/items")           # GET
@app.post("/items")          # POST
@app.put("/items/{id}")      # PUT (パスパラメータ)
@app.delete("/items/{id}")  # DELETE
```

2. POST + Pydantic でデータを受け取る

POSTでJSONボディを受け取る場合は、Pydanticモデルを引数に取る。

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class EchoRequest(BaseModel):
    message: str = Field(min_length=1, max_length=1000)

@app.post("/api/echo")
def echo(req: EchoRequest):
    return {"echo": req.message}
```

ポイント:

- `BaseModel` を継承して型を書くだけでJSONをパースしてくれる
- `Field(...)` でバリデーション(最小・最大長など)
- 不正なリクエストは FastAPI が自動で **422 Unprocessable Entity** を返す

バリデーションの動き

```
class EchoRequest(BaseModel):  
    message: str = Field(min_length=1, max_length=1000)
```

入力	結果
<code>{"message": "hi"}</code>	OK → 200
<code>{"message": ""}</code>	422 (空文字)
<code>{"message": 123}</code>	422 (型違反)
<code>{}</code>	422 (必須フィールド欠落)

「バリデーションを自分で書かなくていい」のがFastAPIの嬉しいところ。

Swagger UI を試してみる

サーバー起動中に `http://localhost:8000/docs` にアクセスすると、FastAPIが自動生成したAPIドキュメント(Swagger UI)が見られる。

- エンドポイント一覧
- リクエスト/レスポンスの型
- ブラウザから直接叩いて試せる

開発中はめちゃくちゃ便利。フロントを書く前にここで挙動を確認できる。

3. CORS と StaticFiles の復習

```
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# static/ ディレクトリを / で配信
app.mount("/", StaticFiles(directory="static", html=True), name="static")
```

- **CORS** = 別オリジンからのfetchを許可する設定。開発中は `*` でOK
- **StaticFiles** = `static/index.html` などを直接ブラウザに返せる

4. フロントからの fetch

ブラウザの JavaScript からサーバーAPIを呼ぶ標準の手段。

```
// GET
const res = await fetch("/api/items");
const data = await res.json();
console.log(data);
```

```
// POST + JSONボディ
const res = await fetch("/api/echo", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ message: "こんにちは" }),
});
const data = await res.json();
```

async / await の復習

`fetch` は **Promise** を返す。 `await` で結果が返ってくるまで待つ。

```
// async関数の中でしか await は使えない
async function sendMessage() {
  const res = await fetch("/api/echo", { ... });
  const data = await res.json();
  console.log(data.echo);
}
```

`addEventListener` の中で `async` を使うには:

```
form.addEventListener("submit", async (event) => {
  event.preventDefault();
  // ここで await が使える
});
```


エラーハンドリングの定石

`fetch` は ネットワークエラーのときしか reject されない。
HTTPステータスが 4xx / 5xx でも `res.ok === false` になるだけ。

```
try {
  const res = await fetch("/api/echo", { ... });
  if (!res.ok) {
    throw new Error(`サーバーエラー: ${res.status}`);
  }
  const data = await res.json();
  // 成功時の処理
} catch (err) {
  // ネットワーク or 上の throw が来る
  console.error(err);
}
```

XSS対策 — `textContent` を使う

サーバーから返ってきた文字列を画面に出すとき、`innerHTML` は危険。
ユーザー入力に `<script>` が混ざっていたら実行されてしまう。

```
// NG: 危ない
element.innerHTML = data.echo;

// OK: テキストとしてだけ扱う
element.textContent = data.echo;
```

「入力信用しない」は前コース第8回(セキュリティ)で扱った原則。
LLMの返答も「他人が書いた文字列」なので同じ扱いをする。

後半: SQLite と演習

5. SQLite のおさらい

- ファイル1個で完結する 軽量データベース
- Pythonには標準で `sqlite3` モジュールが入っている
- このコースの最終形 `chat-app` でも会話履歴の保存に使う

```
import sqlite3

conn = sqlite3.connect("chat.db")
cursor = conn.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS messages (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        role TEXT NOT NULL,
        content TEXT NOT NULL
    )
""")
conn.commit()
conn.close()
```

INSERT / SELECT の基本

```
conn = sqlite3.connect("chat.db")
cursor = conn.cursor()

# 追加
cursor.execute(
    "INSERT INTO messages (role, content) VALUES (?, ?)",
    ("user", "こんにちは"),
)
conn.commit()

# 取得
cursor.execute("SELECT id, role, content FROM messages ORDER BY id")
rows = cursor.fetchall()
for row in rows:
    print(row) # (1, 'user', 'こんにちは')

conn.close()
```

? プレースホルダ を使うのが必須。

`f"... '{user_input}'"` は **SQLインジェクション** の温床。

row_factory で辞書っぽく扱う

タプルではなく 名前でアクセス したいとき。

```
conn = sqlite3.connect("chat.db")
conn.row_factory = sqlite3.Row # ←これ

cursor = conn.cursor()
cursor.execute("SELECT id, role, content FROM messages")
for row in cursor.fetchall():
    print(row["role"], row["content"])
```

chat-app/main.py でもこの形を使う。

読みやすいし、列の順番に依存しないので保守しやすい。

接続は使うたびに `connect` → `close`

毎回の関数で `connect` して、最後に `close` する書き方が一番分かりやすい

```
def get_all_messages():
    conn = sqlite3.connect("chat.db")
    conn.row_factory = sqlite3.Row
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM messages")
    rows = cursor.fetchall()

    conn.close()
    return rows
```

- 関数ごとに自分で開いて、自分で閉じる
- `with` 文を使う書き方もあるが、まずは「明示的に閉じる」習慣をつける

第7回(履歴の永続化)で 再びこの形に出会う。

「あ、第3回でやったやつだ」となれば OK。

6. 演習: echo API + 簡易UI

サーバーが受け取った文字列を **そのまま返す** だけのミニアプリを作る。

仕様:

- `POST /api/echo` を実装
 - リクエスト: `{"message": "..."}`
 - レスポンス: `{"echo": "..."}`
- フロント(`static/index.html` + `app.js`)から `fetch` で呼ぶ
- 返ってきた `echo` を画面に **`textContent`** で 表示する

OpenAI API は使いません。 APIキーも不要。

`export OPENAI_API_KEY=...` も今日はやらなくてOK。

ファイル構成

```
session03/exercise/  
├── main.py           # FastAPI バックエンド  
├── static/  
│   ├── index.html   # 入力フォーム + 応答表示  
│   ├── style.css  
│   └── app.js        # fetch で /api/echo を呼ぶ  
└── README.md
```

- 自分でゼロから書いてもよい
- `session03/exercise/` に完成形があるので、つまったらコピーしてOK

main.py の要点

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field

app = FastAPI()

class EchoRequest(BaseModel):
    message: str = Field(min_length=1, max_length=1000)

@app.post("/api/echo")
def echo(req: EchoRequest):
    return {"echo": req.message}

app.mount("/", StaticFiles(directory="static", html=True), name="static")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

app.js の要点

```
const form = document.getElementById("echo-form");
const input = document.getElementById("message-input");
const responseEl = document.getElementById("response");

form.addEventListener("submit", async (event) => {
  event.preventDefault();
  const message = input.value.trim();
  if (!message) return;

  const res = await fetch("/api/echo", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message }),
  });
  const data = await res.json();
  responseEl.textContent = data.echo; // ← textContent
});
```

動作確認の手順

1. `cd session03/exercise && python main.py`
2. Codespaces のポート 8000 をブラウザで開く
3. 文字列を入力して「送信」を押す
4. 入力した文字列がそのまま表示されればOK

curl でも確認できる:

```
curl -X POST http://localhost:8000/api/echo \  
  -H "Content-Type: application/json" \  
  -d '{"message": "こんにちは"}'  
# => {"echo": "こんにちは"}
```

Swagger UI: `http://localhost:8000/docs`

DevTools で観察する

ブラウザの DevTools (F12) を開いて:

- **Network タブ** で `/api/echo` への POST を確認
 - リクエストペイロード(送信したJSON)
 - レスポンス(返ってきたJSON)
- **Console タブ** でエラーがないか確認
- **要素タブ** で `textContent` で入った文字列を確認

第6回でこの観察スキルが効いてくる。

マルチターン会話で `messages` 配列が伸びていく様子を実際にここで覗く。

本日のまとめ

学んだこと

1. **FastAPI** = デコレータ + Pydantic で簡単にAPIが書ける
2. フロントは `fetch` + `async/await` でAPIを叩く
3. 表示には `textContent` (XSS対策)
4. **SQLite** は `sqlite3` モジュールで簡単に使える / `?` プレースホルダ必須
5. `chat-app` のバックボーンは全部この上に乗る

ここまでが「土台」。来週からこの土台に **LLM** を乗せていく。

次回予告

第4回: OpenAI APIに初めて触れる

ついに LLM をコードから呼ぶ 回。 `pip install openai` → `client.chat.completions.create(...)` でターミナルから対話する CLI スクリプト (`python chat.py`) を作る。 **APIキーは講師から配布** (受講生個人での取得は不要)。

提出物

実習で作成したファイルをフォームから提出してください:

1. `session03/exercise/` の echo API が動いている GitHub のURL
 - 例: `https://github.com/ユーザー名/リポジトリ名/tree/main/session03/exercise`

お疲れ様でした！